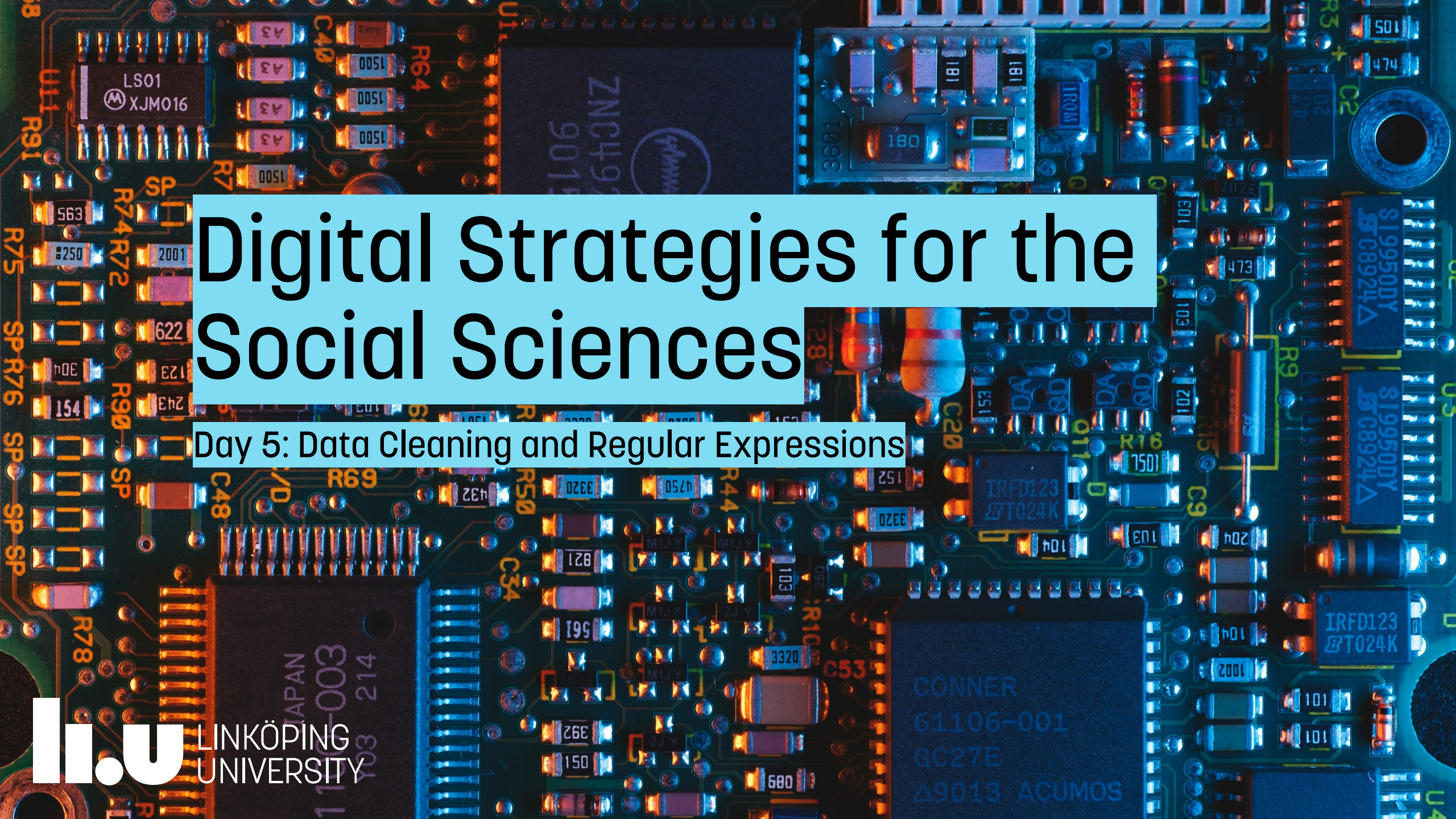




Digital Strategies for the Social Sciences

Day 5: Data Cleaning and Regular Expressions

Today's Agenda

- The last puzzle piece: regular expressions!

Regular Expressions

Pattern Matching from Hell

Regular Expressions

- When you want to extract data from text, you often search for literal strings
- This works well when, say, counting the occurrences of “1984” in a text



Regular Expressions

- When you want to extract data from text, you often search for literal strings
- This works well when, say, counting the occurrences of “1984” in a text
- But what if you only want to find `*any*` year?
- Or variations of color/colour?
- Or email-addresses?

Regular Expressions

- When you want to extract data from text, you often search for literal strings
- This works well when, say, counting the occurrences of “1984” in a text
- But what if you only want to find `*any*` year?
- Or variations of color/colour?
- Or email-addresses?
 - → Regular Expressions!

Regular Expressions

- How do you find a year in a text?

Regular Expressions

- How do you find a year in a text?
- A year is a sequence of four digits

Regular Expressions

- How do you find a year in a text?
- A year is a sequence of four digits

```
text <- "The year is 2026."
year <- ""

for (i in 1:nchar(text)) {
  ch <- substr(text, i, i)
  if (!is.na(as.numeric(ch)) &&
      nchar(year) != 4) {
    year <- paste0(year, ch)
  }
}

year <- as.numeric(year)
print(year)
```

Regular Expressions

- How do you find a year in a text?
- A year is a sequence of four digits
- This is a silly solution.

```
text <- "The year is 2026."
year <- ""

for (i in 1:nchar(text)) {
  ch <- substr(text, i, i)
  if (!is.na(as.numeric(ch)) &&
      nchar(year) != 4) {
    year <- paste0(year, ch)
  }
}

year <- as.numeric(year)
print(year)
```

Regular Expressions

- A regular expression helps you find not exact matches, but *patterns*

```
text <- "The year is 2026."  
regmatches(  
  text,  
  regexpr("\\d{4}", text)  
)
```

```
# outputs: "2026"
```

Regular Expressions

- A regular expression helps you find not exact matches, but *patterns*
 - (However, behind the scenes, your computer will use something similar to the manual solution above)

```
text <- "The year is 2026."  
regmatches(  
  text,  
  regexpr("\\d{4}", text)  
)
```

```
# outputs: "2026"
```

Regular Expressions

- A regular expression helps you find not exact matches, but *patterns*
- With `\d{4}` you match four consecutive digits.

```
text <- "The year is 2026."  
regmatches(  
  text,  
  regexpr("\\d{4}", text)  
)  
  
# outputs: "2026"
```

Regular Expressions

- A regular expression helps you find not exact matches, but *patterns*
- With `\d{4}` you match four consecutive digits.
- You can find variations of color or colour using `colou?r`.
- You can find email addresses using `.+@.+\. [a-z]{2, 10}`

```
text <- "The year is 2026."  
regmatches(  
  text,  
  regexpr("\\d{4}", text)  
)
```

```
# outputs: "2026"
```

Regular Expressions

- A regular expression helps you find not exact matches, but *patterns*
- With `\d{4}` you match four consecutive digits.
- You can find variations of color or colour using `colou?r`.
- You can find email addresses using `.+@.+\.[a-z]{2, 10}`

```
text <- "The year is 2026."  
regmatches(  
  text,  
  regexpr("\\d{4}", text)  
)
```

```
# outputs: "2026"
```

Email RegExp is surprisingly hard,
see: <https://emailregex.com/>

Regular Expressions

in R Code

RegExp in R

- `grep/grepv/grepl`
- `sub/gsub`
- `regexpr/gregexpr`
- `regexec/gregexec`
- `regmatches`
- There are alternatives in tidyverse

RegExp in R

- `grep/grepv/grepl`
- `sub/gsub`
- `regexpr/gregexpr`
- `regexec/gregexec`
- `regmatches`
- There are alternatives in tidyverse
- The “grep” functions “grab” all strings that match the pattern
- It expects a list of strings, not a single one
 - **grep** returns the indices of elements that match
 - **grepv** returns the values of elements that match
 - **grepl** returns a logical vector indicating which elements match

RegExp in R

- `grep/grepv/grepl`
 - `sub/gsub`
 - `regexpr/gregexpr`
 - `regexec/gregexec`
 - `regmatches`
 - There are alternatives in tidyverse
- The “sub” functions substitute things in a string
 - They expect a single string
 - **sub** replaces only the first match with your replacement
 - **gsub** replaces all

RegExp in R

- `grep/grepv/grepl`
 - `sub/gsub`
 - `regexpr/gregexpr`
 - `regexec/gregexec`
 - `regmatches`
 - There are alternatives in tidyverse
- The `regexpr` functions search the input for matches, and return information on them
 - Like with “`sub`”, the “`g`” indicates “all” matches instead of the first one (`g` = “global”)
 - They return the index of the match, and its length

RegExp in R

- `grep/grepv/grepl`
 - `sub/gsub`
 - `regexpr/gregexpr`
 - `regexec/gregexec`
 - `regmatches`
 - There are alternatives in tidyverse
- The exec-type functions execute a regular expression
 - For simple expressions, their output is comparable to `regexpr`, but they give you even more information on each match (incl. capturing groups)

RegExp in R

- `grep/grepv/grepl`
 - `sub/gsub`
 - `regexpr/gregexpr`
 - `regexec/gregexec`
 - `regmatches`
 - There are alternatives in tidyverse
- `regmatches` is a utility function that takes the original string plus the result of `regexpr` or `regexec` to extract the *values* of matches from that text
 - `regexpr` and `regexec` only provide information, not the values!

RegExp in R

- `grep/grepv/grepl`
 - `sub/gsub`
 - `regexpr/gregexpr`
 - `regexec/gregexec`
 - `regmatches`
 - There are alternatives in tidyverse
- Since regular expressions are very versatile, you can also use them in various tidyverse results
 - Many functions (incl. in other libraries) support regular expressions, so make sure to check the documentation!

Regular Expressions

Rules and Applications

RegExp Rules

- Regular Expressions are *yet another language* that allows you to formulate queries to find *patterns* in text

RegExp Rules

- Let's start simple: Variations of a word
 - color and colour
 - http and https

RegExp Rules

- Let's start simple: Variations of a word
 - color and colour
 - http and https



```
1 /colou?r/
```

```
2 /https?/
```

RegExp Rules

- Let's start simple: Variations of a word
 - color and colour
 - http and https

Note: This is JavaScript syntax for regular expressions. I find this clearer and easier to parse. The // around the expression are not part of the regular expression. In R, you need to use strings and escape backslashes.



```
1 /colou?r/-  
2 /https?/
```

RegExp Rules

- Let's start simple: Variations of a word
 - color and colour
 - http and https
- The ? means: “The previous character can occur once, or not at all”



```
1 /colou?r/
```

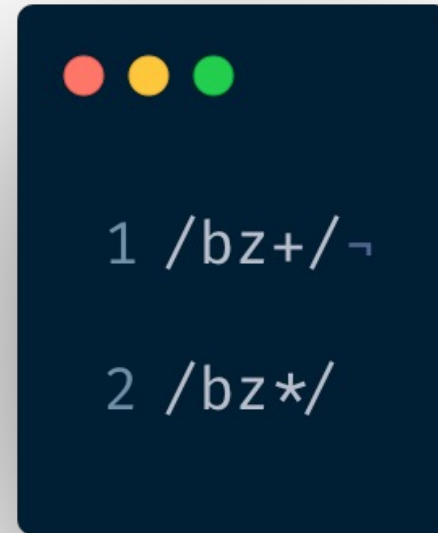
```
2 /https?/
```

RegExp Rules

- Next: What if we don't know how many characters there are?
 - `bzzzzzz`

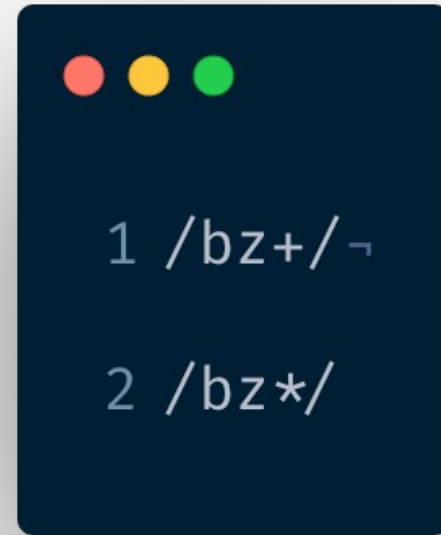
RegExp Rules

- Next: What if we don't know how many characters there are?
 - bzzzzzz



RegExp Rules

- Next: What if we don't know how many characters there are?
 - bzzzzzz
- The + means: “The preceding character must be present at least once, but as many times as possible”
- The * means: “The preceding character is optional, but if it's present, as many times as possible



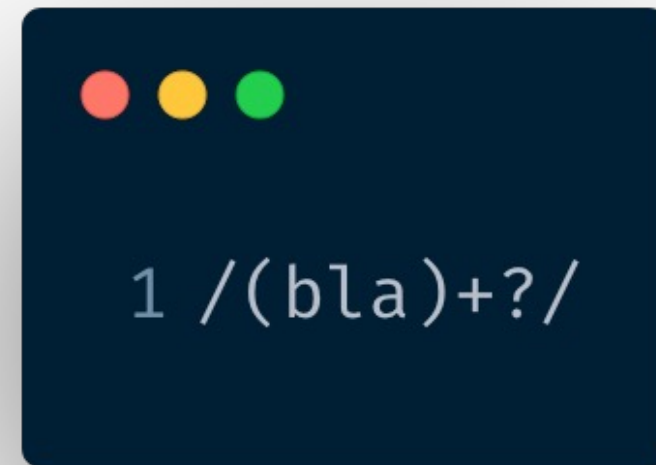
RegExp Rules

- Next: What if we don't know how many characters there are?
 - bzzzzzz
- By default, + and * are “greedy”: They want to match as much of the search string as possible
- A question mark afterwards tells your RegExp that you want to match as few as possible.



RegExp Rules

- This makes more sense if you want to match more than one character
 - This matches the sequence “bla” multiple times
- String: “blablablablablablabla”
 - With question mark:
Matches 8x “bla”
 - Without question mark:
Matches 1x “blablablabla...”



RegExp Rules

- What if you wanted to match words?

RegExp Rules

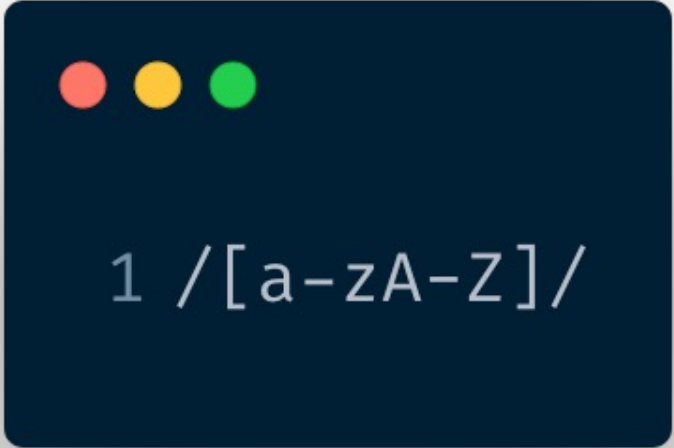
- What if you wanted to match words?



```
1 /[a-zA-Z]/
```

RegExp Rules

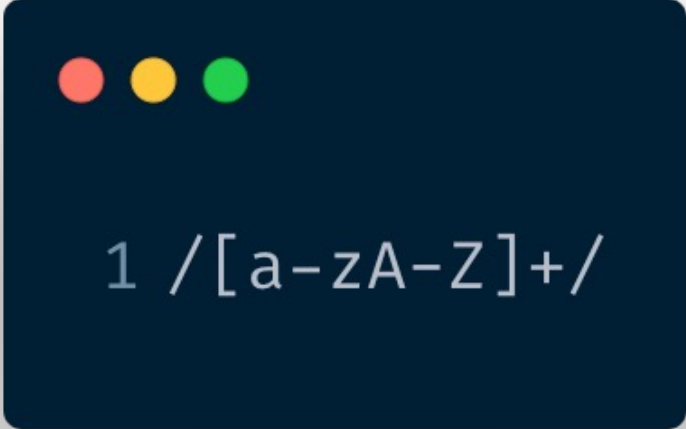
- What if you wanted to match words?
 - What's missing here?



```
1 /[a-zA-Z]/
```

RegExp Rules

- What if you wanted to match words?
 - What's missing here?
 - Without the + it will match individual letters, not words



```
1 /[a-zA-Z]+/
```

RegExp Rules

- Sometimes, writing out all the letters that you want to look for is tedious.
 - That's what “character classes” are for
 - `\w`: “Word” characters
 - `\d`: Numbers
 - `\s`: Any whitespace (incl. tabs)

Important: In R, regular expressions are strings,
and so you need to escape backslashes!
→ `"\\d+"`



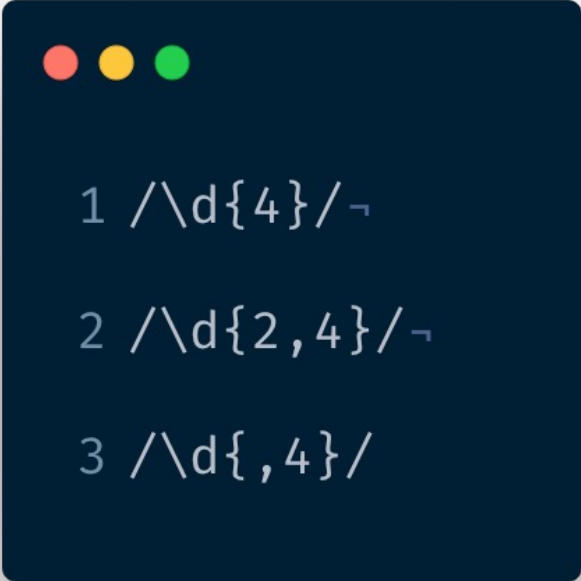
RegExp Rules

- Sometimes, writing out all the letters that you want to look for is tedious.
 - That's what “character classes” are for
 - `\w`: “Word” characters
 - `\d`: Numbers
 - `\s`: Any whitespace (incl. tabs)
 - `\W`: Not-word characters
 - `\D`: Non-numbers
 - `\S`: Non-whitespace



RegExp Rules

- If you want to specify a number of repeats, use curly brackets



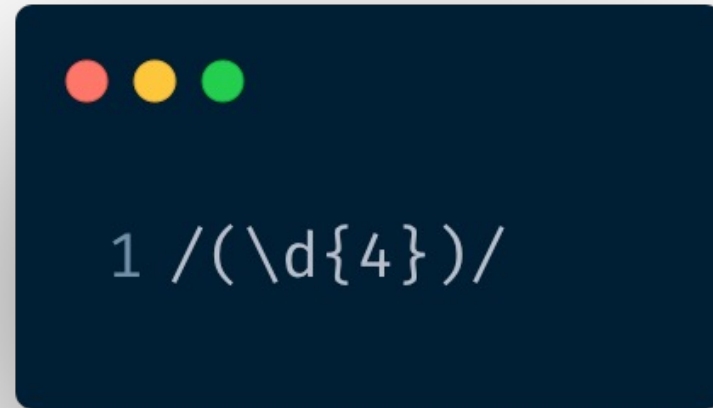
```
1 /\d{4}/
```

```
2 /\d{2,4}/
```

```
3 /\d{,4}/
```

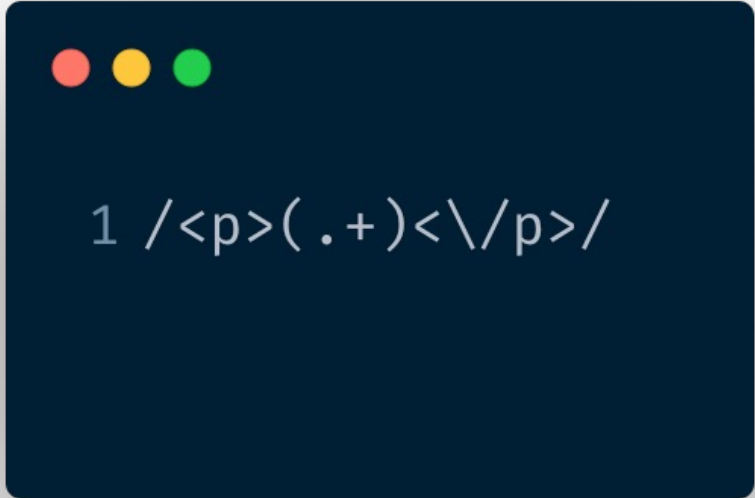
RegExp Rules

- What if you do not just want to *search* for patterns, but extract them?
 - Use “capturing groups”



RegExp Rules

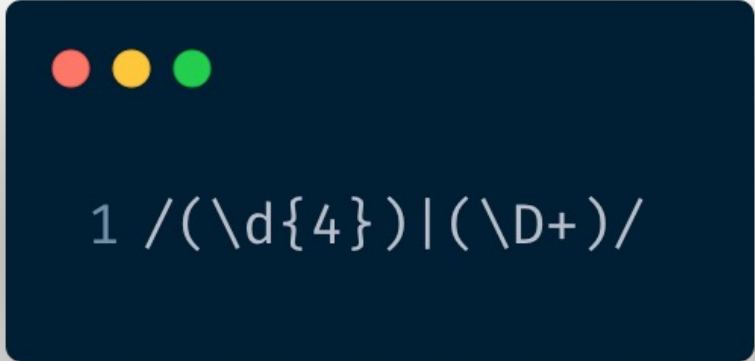
- How to match HTML tags:
 - Use . (the wildcard)



```
1 /<p>( .+ )<\/p>/
```

RegExp Rules

- Find alternatives
 - The vertical line (“pipe”) specifies two alternatives



```
1 /(\d{4})|(\D+)/
```

RegExp Rules

- Find things at the start or end of a string
 - ^ for the beginning
 - \$ for the end
 - → You don't have to specify both!



```
1 /^https?/  
2 /\.jpe?g$/
```

Helpers and Utilities

So that RegExp isn't too hard

regex101.com

The screenshot displays the regex101.com interface. The browser address bar shows 'regex101.com'. The page title is 'regular expressions 101'. The left sidebar contains navigation links: 'Save & Manage Regex', 'Save & Share', 'Editor', 'Flavor: PCRE2 (PHP)', 'Match', 'Substitution', 'List', 'Unit Tests', 'Tools', 'Code Generator', 'Regex Debugger', 'Export Matches', 'Benchmark Regex', and 'Format Regex'. The main area shows the 'Regular Expression' input field with the regex `/(?:\d{4})|(\D+)/gm`. Below it, the 'Test String' is a paragraph of text about the Chronos Project, with years highlighted in green. The 'Matches' section shows two matches: 'Match 1' (0-37) and 'Match 2' (37-41). The 'Explanation' section on the right provides a detailed breakdown of the regex components, including the 1st Alternative `(?:\d{4})` and the 2nd Alternative `(\D+)`. The 'Match Information' section shows the match details for 'Match 1' and 'Match 2'. The 'Quick Reference' section at the bottom right provides a search bar and a list of common tokens.

regular expressions 101

Support Regex101 Social Info

Regular Expression `/(?:\d{4})|(\D+)/gm` Matches: 21 Steps: 106 Time: 1 ms

Test String

The Chronos Project began quietly in 1985, an ambitious attempt to map temporal anomalies using obsolete satellite technology. Initial funding dried up, but the core team persevered, driven by the theory that reality itself was subject to cyclical shifts. By 1992, they secured a breakthrough, proving that certain historical events left faint, measurable echoes in the present. This discovery spurred a massive international effort, culminating in the establishment of the Temporal Preservation Bureau in 2001. The early years were marked by intense secrecy, but by 2015, the first stable time-jump mechanism was successfully tested, opening up unimaginable possibilities. The next decade saw rapid advancement; by 2022, researchers were already planning voyages to the deep past. A major setback occurred in 2030 when a catastrophic energy surge nearly destroyed the primary research facility, forcing a complete overhaul of the system. Despite the difficulties, progress continued relentlessly. A crucial stabilization technique was finally achieved in 2045, allowing for safer, shorter jumps. By 2050, the Bureau shifted its focus from observation to subtle intervention. The ultimate test of their theories came in 2061, when they successfully navigated a historical divergence, confirming their fears and their hopes. Looking ahead, the team was already preparing for the next phase, aiming for a stable timeline by 2075.

Pos: 11,428

Explanation

- 1st Alternative `(?:\d{4})`
 - 1st Capturing Group `(\d{4})`
 - `\d{4}` matches a digit (equivalent to `[0-9]`)
 - `{4}` matches the previous token exactly 4 times
- 2nd Alternative `(\D+)`
 - 2nd Capturing Group `(\D+)`
 - `\D` matches any character that's not a digit

Match Information

Match	Start	End	Text
Match 1	0	37	The Chronos Project began quietly in
Group 2	0	37	The Chronos Project began quietly in
Match 2	37	41	1985
Group 1	37	41	1985

Quick Reference

Search reference

- All Tokens
- Common Tokens ☒ A character except a, b ... `[^abc]`
- General Tokens
- anchors
- A character in the range ... `[a-z]`
- A character not in the ra... `[^a-z]`
- A character in the ran... `[a-zA-Z]`

regular-expressions.info



Other Text Cleaning

You don't always need regular expressions

Trim text

- Removes whitespace from the start and finish
 - `trim(text)`

Normalize text

- Replace, e.g., any whitespace character with a simple space
 - `gsub("\\s+", " ", text)`
- You can make use of capturing groups to normalize text, too:

```
text ← c("Bourdieu, Paris 1932 –  
Paris 2002")
```

```
gsub(".*(\\d{1,4}).*(\\d{1,4})",  
"\\1 – \\2", text)
```

```
# Returns "1932 – 2002"
```

Split text

- Splits text at certain points
 - Can be used to parse ISO dates (YYYY-MM-DD)
 - `stringi::str_split_1(text, "-")`

Any questions?

Thank you!